



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Head TA
Nicolas Reed

Caches III: Direct Mapped

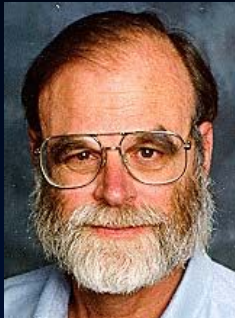
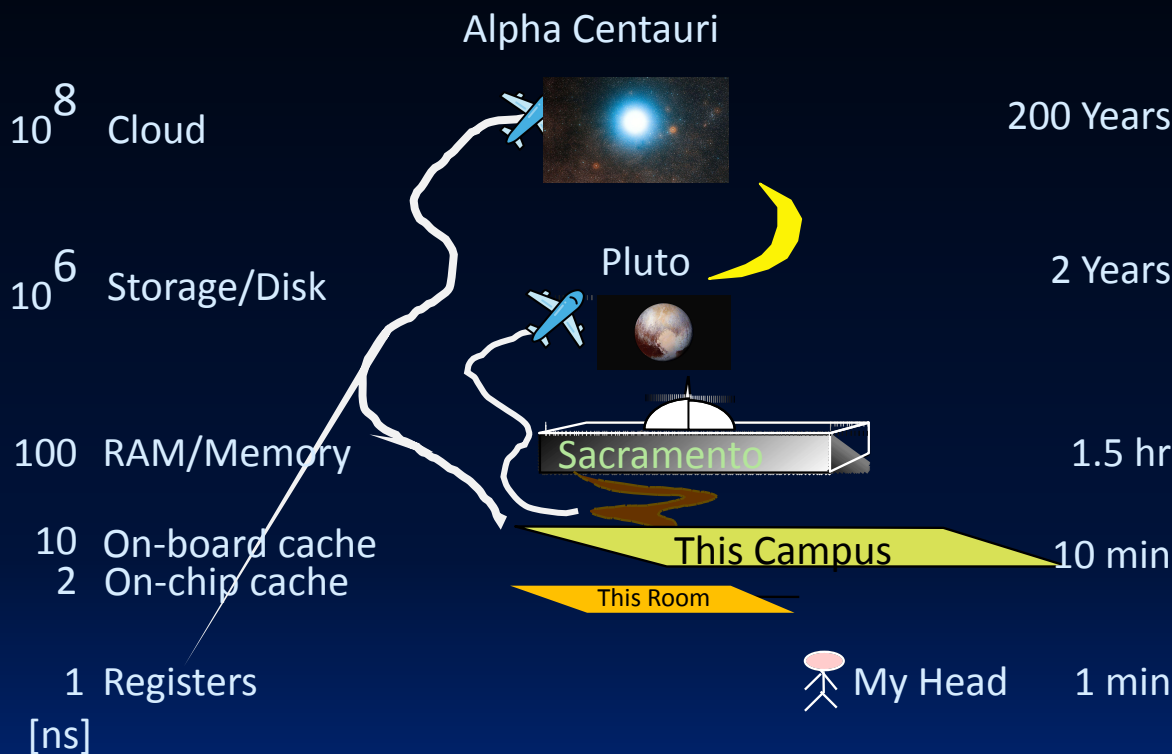


Assistant
Teaching Professor
Lisa Yan

Yan, SP26

Great Idea #3: Principle of Locality / Memory Hierarchy

Storage Latency Analogy: How Far Away is the Data?



Jim Gray
 1998 Turing Award
 B.S. Cal 1966
 Ph.D. Cal 1969



Line Replacement Policies

- Line Replacement Policies
 - Write Policies
 - Direct Mapped Cache
 - Direct Mapped Cache Analysis
-
- Types of Misses



Fully Associative Cache with LRU policy

- By the end of instr 5, cache is **warm**.
- 1. Load byte 0x43F 0x10F, 0x3
 - 2. Load byte 0x5E2 0x178, 0x2
 - 3. Load word 0x824 0x209, 0x0
 - 4. Load word 0x5E0 0x178, 0x0
 - 5. Load word 0x524 0x149, 0x0
 6. Load byte 0x972 0x25C, 0x2

Val id	LRU	Tag	Data			
			11	10	01	00
1	3	0x10F	...	...	...	...
1	1	0x178	...	...	...	...
1	2	0x209	...	...	...	...
1	0	0x149	...	...	...	...

Suppose that LRU = 0 means **most** recently used, and 3 means **least** recently used.

After the end of instruction 5, what are the LRU tags on each row?



A Warmed up Cache Can Still Miss

- By the end of instr 5, cache is **warm**.
- Load byte 0x43F 0x10F, 0x3
 - Load byte 0x5E2 0x178, 0x2
 - Load word 0x824 0x209, 0x0
 - Load word 0x5E0 0x178, 0x0
 - Load word 0x524 0x149, 0x0
 - Load byte 0x972 0x25C, 0x2

Val id	LRU	Tag	Data			
			11	10	01	00
1	0	0x25C	...	...	...	...
1	2	0x178	...	...	...	...
1	3	0x209	...	...	...	...
1	1	0x149	...	...	...	...

- Cache miss!**
- Load into cache the 4-byte line from 0x970 to 0x973. Mark valid bit. Update line replacement policy fields (e.g., LRU).
- Read byte at 0x2 offset, return to processor.

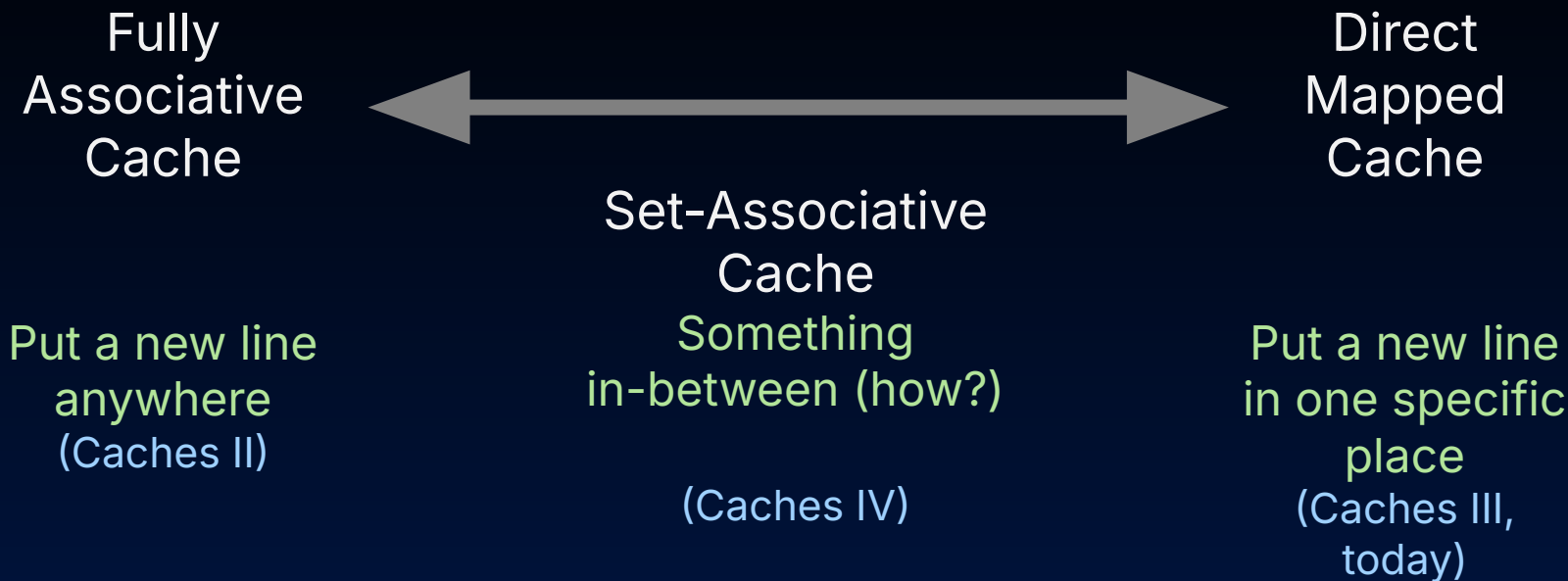
Line Replacement Policies

- Least Recently Used (LRU)
 - Replace the entry that has not been used for the longest time, i.e., has the oldest previous access.
 - Pro: **Temporal locality!**
 - recent past use implies likely future use
 - Con: Complicated hardware to keep track of access history
- In practice: First In, First Out (FIFO)
 - Replace the oldest line in the set ("queue").
- Other policies (see in notes):
 - Most Recently Used (MRU)
 - Random
 - Last In, First Out (LIFO) - "stack"

LRU is ideal for **temporal locality** but in practice, FIFO is good enough.



Cache Design: Placement Policies, Soon





Write Policies

- Line Replacement Policies
 - Write Policies
 - Direct Mapped Cache
 - Direct Mapped Cache Analysis
-
- Types of Misses



Stores: How to write back to memory?

- By the end of instr 5, cache is warm.

1. Load byte 0x43F 0x10F, 0x3
2. Load byte 0x5E2 0x178, 0x2
3. Load word 0x824 0x209, 0x0
4. Load word 0x5E0 0x178, 0x0
5. Load word 0x524 0x149, 0x0
6. Load byte 0x972 0x25C, 0x2
7. Store byte 0x524 0x149, 0x0

Val id	LRU	Tag	Data			
			11	10	01	00
1	0	0x25C	...	...	...	...
1	2	0x178	...	...	...	...
1	3	0x209	...	...	...	...
1	1	0x149	...	...	...	...

Cache hit! ...and then?
How to handle stores?



Write-through vs. Write-back Policies

- **Store** instructions write to memory, which **changes values**.
- Hardware needs to ensure that cache and memory **"sync"** their data.

Val id	LRU	Tag	Data			
			11	10	01	00
1	0	0x25C	...	...	...	...
1	2	0x178	...	...	...	...
1	3	0x209	...	...	...	...
1	1	0x149	...	...	...	...

Store byte 0x524 (0x149, 0x0)

- **Write-through:**
 - Write to the cache and memory at the same time.
- **Write-back:**
 - Write data in cache for now.
 - When this line gets replaced from the cache (and "back" to memory), write to memory.

What does Write-Back Look Like? Dirty Bit

- **Write-back:**
 - Write data in cache and set a **dirty bit** to 1.
 - When this line gets replaced from the cache (and "back" to memory), write to memory.
- Fully Associative Cache w/ LRU, Write-Back

Val id	Dir ty	LRU	Tag	Data			
				11	10	01	00
1	0	0	0x25C	...	...	...	...
1	0	2	0x178	...	...	...	...
1	0	3	0x209	...	...	...	...
1	1	1	0x149	...	...	...	...

Store byte 0x524 (0x149, 0x0)

Cache hit w/**write-back**:
update cache line, and **wait until
this line is replaced** before
writing back to memory



Write-through vs. Write-back Policies

Write policies have tradeoffs:

- **Write-through:**
 - Write to the cache and memory at the same time.
 - (more writes to memory → longer AMAT)
- **Write-back:**
 - Write data in cache and set a **dirty bit** to 1.
 - When this line gets replaced from the cache (and "back" to memory), write to memory.

Val id	LRU	Tag	Data			
			11	10	01	00
1	0	0x25C	...	...	...	...
1	2	0x178	...	...	...	...
1	3	0x209	...	...	...	...
1	1	0x149	...	...	...	...

Store byte 0x524 (0x149, 0x0)

simple to
implement

(typically) lower
traffic to memory

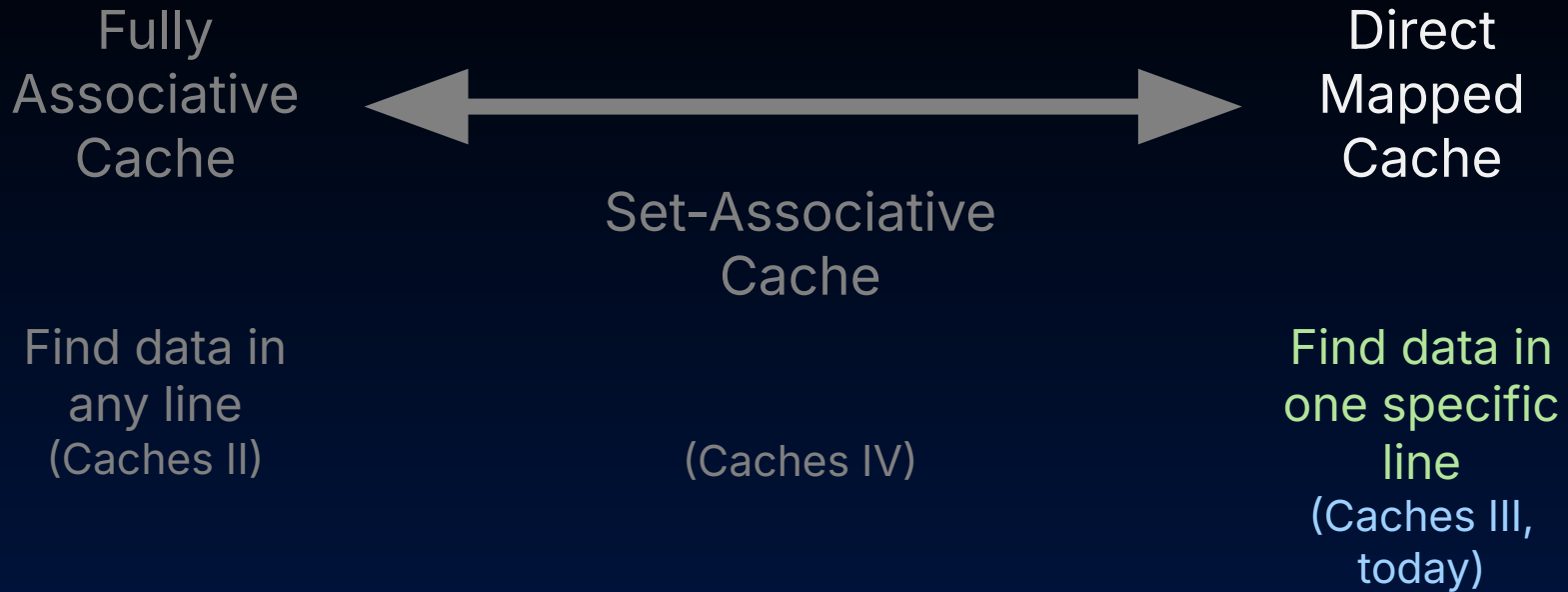


Direct Mapped Cache

- Line Replacement Policies
 - Write Policies
 - Direct Mapped Cache
 - Direct Mapped Cache Analysis
-
- Types of Misses



Cache Design: Placement Policies



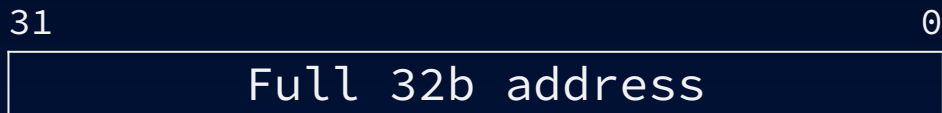
Fully associative caches need expensive hardware.



Direct Mapped Cache

- **Placement policy:** The data at a memory address can be stored at exactly one possible line in the cache.

flags	Tag	Data			
		11	10	01	00
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...



How do we identify this location from the memory address?

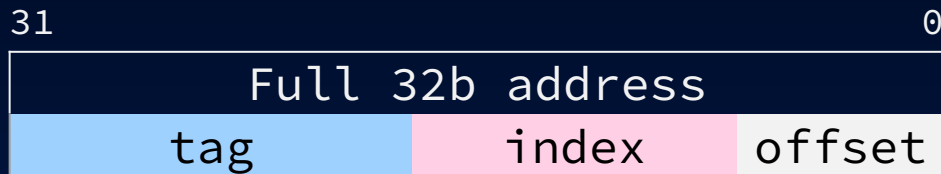


Direct Mapped Cache

- Placement policy:** The data at a memory address can be stored at exactly one possible line in the cache.

indices { 0, 1, 2, 3

flags		Tag	Data			
			11	10	01	00
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...

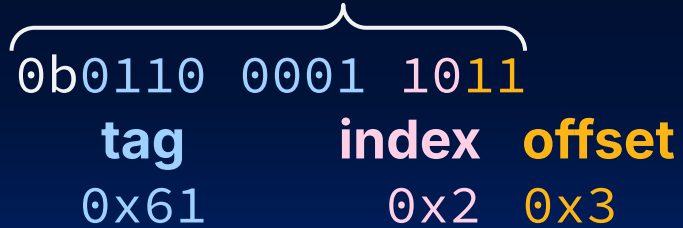


tag to connect line to memory address

index to select line in cache

byte offset within line

Example: 0x61B





Direct Mapped Caches: 4B Cache

flags		Tag	Data
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

- 4B Direct Mapped Cache (line size 1B)

Line starting...	maps to...	has tag...
○ @ 0x0	index 0	000000
○ @ 0x1	index 1	000000
○ @ 0x2	index 2	000000
○ @ 0x3	index 3	000000

Different addresses get **same line index** but **different tags**.

Tag: 000000 000001

(assume 8 bit address)

0x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F

Yan, SP26

Direct Mapped Caches: 4B Cache

flags		Tag	Data
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

Lines with **same line index** replace each other.

- 4B Direct Mapped Cache (line size 1B)

Line starting...	maps to...	has tag...
○ @ 0x0	index <u>0</u>	000000
○ @ 0x1	index 1	000000
○ @ 0x2	index 2	000000
○ @ 0x3	index 3	000000
○ @ 0x4	index <u>0</u>	000001
○ @ 0x8	index <u>0</u>	000010
○ @ 0xC	index <u>0</u>	000011
○ etc.		

0x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F

Yan, SP26

Direct Mapped Caches: 8B Cache

flags		Tag	Data	
			1	0
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

- 8B Direct Mapped Cache (line size 2B)

Line starting...	maps to...	has tag...
○ @ 0x0	index 0	00000
○ @ 0x2	index 1	00000
○ @ 0x4	index 2	00000
○ @ 0x6	index 3	00000
○ @ 0x8	index 0	00001
○ @ 0x10	index 0	00010
○ @ 0x18	index 0	00011
○ etc.		

Tag: 00000

0x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F

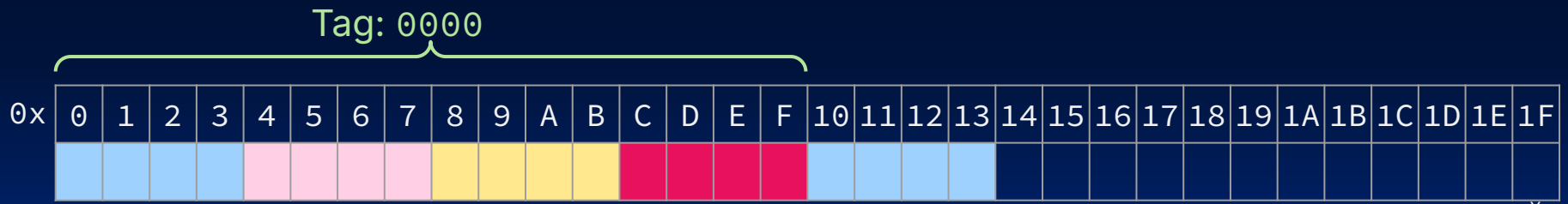
Yan, SP26

Direct Mapped Caches: 16B Cache

flags		Tag	Data			
			11	10	01	00
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...

- 16B Direct Mapped Cache (line size 4B)

Line starting...	maps to...	has tag...
○ @ 0x0	index 0	0000
○ @ 0x4	index 1	0000
○ @ 0x8	index 2	0000
○ @ 0xC	index 3	0000
○ @ 0x10	index 0	0001
○ etc.		



Yan, SP26



Direct Mapped Cache

- **Placement policy:** Each memory address is associated with exactly one possible line in the cache.

flags		Tag	Data			
			11	10	01	00
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...



tag to connect line
to memory
address

index to select
line in cache

byte offset
within line

Unlike FA caches, DM caches have an **index** to identify the **only** cache line for this memory address.

Fill in the blank: Direct Mapped, 12 bit addresses

Suppose we have the below direct mapped cache for **12 bit** addresses.

- | | A. | B. | C. | D. | E. | F. |
|---|----|----|----|----|----|-------|
| 1. What is the line size / block size, in bytes? | 2 | 4 | 8 | 10 | 16 | Other |
| 2. What is the capacity, in bytes? | 2 | 4 | 8 | 10 | 16 | Other |
| 3. For a 12b address, how many bits is the byte offset? | 2 | 4 | 8 | 10 | 16 | Other |
| 4. For a 12b address, how many bits is the index? | 2 | 4 | 8 | 10 | 16 | Other |
| 5. For a 12b address, how many bits is the tag? | | | | | | |

flags	Tag	Data			
		11	10	01	00

Suppose we have the below direct mapped cache for **12b** addresses.

	A.	B.	C.	D.	E.	F.
1. What is the block size / line size, in bytes?	2	4	8	10	16	Other
2. What is the capacity, in bytes?	2	4	8	10	16	Other
3. For a 12b address, how many bits is the byte offset?	2	4	8	10	16	Other
4. For a 12b address, how many bits is the index?	2	4	8	10	16	Other
5. For a 12b address, how many bits is the tag?						





Terminology for Direct Mapped, 12b addresses

1. Line size / block size:
 - # bytes per cache line **4 bytes**
2. Capacity
 - Total # data bytes that can be stored in a cache 4×4 bytes = **16 bytes**
3. Offset

Tag	Data			
	11	10	01	00

- Identifies byte address of data stored at a given cache line $\log_2(\text{line size}) = \mathbf{2 \text{ bits}}$

4. Index
 - Selects cache line $\log_2(\# \text{ lines}) = \mathbf{2 \text{ bits}}$

5. Tag
 - Identifies data stored at a given cache line to a given memory address $\# \text{ address bits} - \# \text{ offset bits} - \# \text{ index bits} = \mathbf{8 \text{ bits}}$

memory address

11	4	3	2	1	0

tag

offset
index



Direct Mapped Cache Analysis

- Line Replacement Policies
 - Write Policies
 - Direct Mapped Cache
 - Direct Mapped Cache Analysis
-
- Types of Misses



Example: Direct Mapped Cache

Suppose we have the following direct mapped cache, for 12 bit addresses.

Load byte 0xFE2

Compute T/I/O

0b1111 1110 0010
tag index offset
0xFE 0x0 0x2

val id	dir ty	Tag	Data			
			11	10	01	00
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...

memory address

11	4	3	2	1	0

tag index offset



Warming up the Direct Mapped Cache

- Suppose we have the following direct mapped cache, for 12b addresses.

Load byte 0xFE2

0b1111 1110 0010 0xFE, 0x0, 0x2
tag index offset

val id	dir ty	Tag	Data			
			11	10	01	00
1	0	0xFE	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...

- Cache miss!** Tag 0xFE is not valid at index 0.
- Load into cache the **4-byte line** from 0xFE0 to 0xFE3.
Mark **valid bit**.
- Read byte at **0x2 offset** and return to processor.

0b1111 1110 0000 ← 0xFE0
0b1111 1110 0011 ← 0xFE3



Warming up the Direct Mapped Cache

- Suppose we have the following direct mapped cache, for 12b addresses.

Store byte 0x61C

0b0110 0001 1100 0x61, 0x3, 0x0
tag index offset

val id	dir ty	Tag	Data			
			11	10	01	00
1	0	0xFE	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...
1	1	0x61	...	...	...	...

- Cache miss!** Tag 0x61 is not valid at index 3.
- Load into cache the **4-byte line**.
Mark **valid bit**.
- Write byte at **0x0 offset**.
Mark **dirty bit**.



Warming up the Direct Mapped Cache

- Suppose we have the following direct mapped cache, for 12b addresses.

Load byte 0x61B

0b0110 0001 1011 0x61, 0x2, 0x3
tag index offset

val id	dir ty	Tag	Data			
			11	10	01	00
1	0	0xFE	...	...	...	...
0	...	...	...	...	...	...
1	0	0x61	...	...	...	...
1	1	0x61	...	...	...	...

- Cache miss!** Tag 0x61 is not valid at index 2.
- Load into cache the **4-byte line**.
Mark **valid bit**.
- Read byte at **0x3 offset**
and return to processor.

Warming up the Direct Mapped Cache

- Suppose we have the following direct mapped cache, for 12b addresses.

Load byte 0xCAD

$\overbrace{0b1100\ 1010\ 1101}^{0xCA, 0x3, 0x1}$
 tag index offset

val id	dir ty	Tag	Data			
			11	10	01	00
1	0	0xFE	...	...	...	...
0	...	...	...	...	...	...
1	0	0x61	...	...	...	...
1	0	0xCA	...	...	...	...

- Cache miss!** Tag 0xCA is not valid at index 3.
- Evict current line at index 3.
Write back data.
- Load into cache the **4-byte line**.
Mark **valid bit**.
- Read byte at **0x1 offset**
and return to processor.



Direct Mapped Cache

- Placement policy:** The data at a memory address can be stored at exactly one possible line in the cache.

flags		Tag	Data			
			11	10	01	00
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...

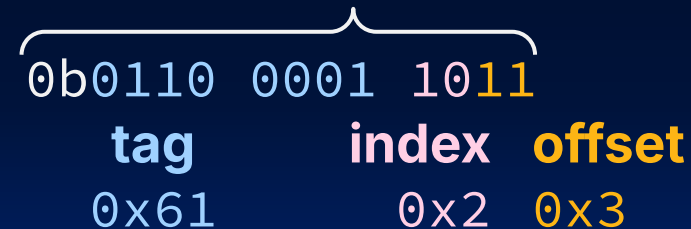


tag to connect line to memory address

index to select line in cache

byte offset within line

Example: 0x61B





Direct Mapped: Policies

What policies can be implemented for a **direct-mapped cache**?
Select all that apply.

1. Write Policy

- A.** Write-through
(memory access per write)
- B.** Write-back
(dirty bit, write to memory
on replacement)

2. Line Replacement Policy

- A.** Least Recently Used
- B.** Most Recently Used
- C.** FIFO
- D.** Random
- E.** None of the Above

Direct Mapped: Policies Solution

What policies can be implemented for a **direct-mapped cache**?
Select all that apply.

1. Write Policy

- A.** Write-through
(memory access per write)
- B.** Write-back
(dirty bit, write to memory
on replacement)

2. Line Replacement Policy

- A.** Least Recently Used
- B.** Most Recently Used
- C.** FIFO
- D.** Random
- E.** None of the Above

In direct mapped caches, there is only ever one line to replace—the existing line with matching index.

Cache Design: Placement Policies, Summary

- Fully Associative
 - **Placement policy:** Data at any memory address can be associated with any cache line
 - **Expensive hardware:** check all lines
 - **Write policies:** write-back, write-through
 - Must decide **replacement policy** (LRU, MRU, FIFO, etc.)
- Direct Mapped
 - **Placement policy:** Each memory address is associated with exactly one possible line in the cache.
 - **Simpler hardware:** check one line
 - **Write policies:** write-back, write-through
 - No **replacement policy**, because we know which line to replace
 - Has conflict misses (see soon)

For the above reasons, smaller caches are generally fully associative.



(pause)



Looking Ahead

- CS 61C: Lectures 26-29, Caches I-IV
- [EECS 151](#) (Digital/IC Design): Reveals the circuitry behind caches
- EECS 151LA (ASIC Lab): [Design your own cache in SystemVerilog](#)
- [CS 152](#) (Comp Arch): Explores advanced cache implementations
- [CS 162](#) (OS): Introduces software and hardware caches



Types of Misses

- Line Replacement Policies
 - Write Policies
 - Direct Mapped Cache
 - Direct Mapped Cache Analysis
-
- Types of Misses

Types of Misses

- **Compulsory Miss**

- Caused by the first access to data that has never been in the cache.

- **Capacity Miss**

- Caused when the cache cannot contain all the lines needed during the execution of a program.
- Occur when lines were in the cache, replaced, and later retrieved.

- **Conflict Miss**

- Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

In this class, we will only distinguish between **compulsory** and **non-compulsory** misses.

“Non-compulsory” miss

Types of Misses

- **Compulsory Miss**

- Caused by the first access to data that has never been in the cache.

- **Capacity Miss**

- Caused when the cache cannot contain all the lines needed during the execution of a program.
- Occur when lines were in the cache, replaced, and later retrieved.

- **Conflict Miss**

- Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

Which types of misses can occur for Fully Associative caches (FA)? For Direct Mapped caches (DM)? Select all that apply.

- A. FA: Compulsory
- B. DM: Compulsory
- C. FA: Capacity
- D. DM: Capacity
- E. FA: Conflict
- F. DM: Conflict
- G. None of the above

L17: Which types of misses can occur for Fully Associative caches (FA)? For Direct Mapped caches (DM)? Select all that apply.

Compulsory Miss

- Caused by the first access to a block that has never been in the cache

Capacity Miss

- Caused when the cache cannot contain all the blocks needed during the execution of a program
- Occur when blocks were in the cache, replaced, and later retrieved

Conflict Miss

- Multiple blocks compete for the same location in the cache, even when the cache has not reached full capacity.

A. FA: Compulsory

0%

B. DM: Compulsory

0%

C. FA: Capacity

0%

D. DM: Capacity

0%

E. FA: Conflict

0%

F. DM: Conflict

0%

G. None of the above

0%



Types of Misses

- **Compulsory Miss**

- Caused by the first access to data that has never been in the cache.

- **Capacity Miss**

- Caused when the cache cannot contain all the lines needed during the execution of a program.
- Occur when lines were in the cache, replaced, and later retrieved.

- **Conflict Miss**

- Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

Which types of misses can occur for Fully Associative caches (FA)? For Direct Mapped caches (DM)? Select all that apply.

- A.** FA: Compulsory
- B.** DM: Compulsory
- C.** FA: Capacity
- D.** DM: Capacity
- ~~**E.** FA: Conflict~~
- F.** DM: Conflict
- G.** None of the above



Types of Misses

- Compulsory Miss
 - Caused by the first access to data that has never been in the cache.
- Capacity Miss
 - Caused when the cache cannot contain all the lines needed during the execution of a program.
 - Occur when lines were in the cache, replaced, and later retrieved.
- Conflict Miss
 - Multiple lines compete for the same location in the cache, even when the cache has not reached full capacity.

In this class, we will only distinguish between **compulsory** and **non-compulsory misses**.

- Occurs in direct mapped caches, as well as set associative caches (more later).
- Misses of this type would not occur in a fully associative cache with similar specs.



How to categorize misses

[reference]

- Run an address trace against a set of caches.
 - (thanks Prof. Kubiawicz for the algorithm).
- 1. First, consider an infinite-size, fully-associative cache. For every miss that occurs now, consider it a **compulsory miss**.
- 2. Next, consider a finite-sized cache (of the size you want to examine) with full-associativity. Every miss that is not in #1 is a **capacity miss**.
- 3. Finally, consider a finite-size cache with finite-associativity. All of the remaining misses that are not #1 or #2 are **conflict misses**.

In this class, we will only distinguish between **compulsory** and **non-compulsory misses**.



[Extra] Practice

- Line Replacement Policies
 - Write Policies
 - Direct Mapped Cache
 - Direct Mapped Cache Analysis
-
- Types of Misses



Direct Mapped Cache (write-back)

Suppose the 4-line, 4B line size cache below starts cold.

1. Load byte 0xFE2
2. Load byte 0xFE8
3. Load word 0xFE9
4. Load word 0xDF9
5. Load byte 0xFE8

1. What is the resulting state of the cache?
2. What misses occur, and are they compulsory or non-compulsory?

val id	dir ty	Tag	Data			
			11	10	01	00
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...

Yan, SP26



Direct Mapped Cache (write-back)

Suppose the 4-line, 4B line size cache below starts cold.

1. Load byte 0xFE2 0b1111 1110 0010 0xFE, 0x0, 0x2
2. Load byte 0xFE8 0b1111 1110 1000 0xFE, 0x2, 0x0
3. Load word 0xFE9 0b1111 1110 1001 0xFE, 0x2, 0x1
4. Load word 0xDF9 0b1101 1111 1001 0xDF, 0x2, 0x1
5. Load byte 0xFE8 0b1111 1110 1000 0xFE, 0x2, 0x0

1. What is the resulting state of the cache?
2. What misses occur, and are they compulsory or non-compulsory?

val id	dir ty	Tag	Data			
			11	10	01	00
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...
0	...	...	...	...	...	...

Yan, SP26